



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Készítette:	Csurgó Márton
Neptun-kód:	HR6BO1
Szakirány:	Információs rendszerek
E-mail cím:	csurgomarton@gmail.com
Konzulens(ek):	Dr. Toka László
E-mail címe(ik):	toka.laszlo@vik.bme.hu

**Téma címe: Stabil párosítási problémák megoldása
Gráf Neurális Hálók segítségével**

Feladat

A stabil párosítás több problémát foglal magába, a problémák a rájuk szabott kényszerekben különböznek egymástól. A célom az, hogy a problémákban rejlő közös matematikai struktúrákra és mintázatokra rátanuljon egy Gráf Neurális Hálózat. A betanított modell képes legyen felismerni ezeket a struktúrákat és a modell által biztosított információ segítségével stabil párosításokat alkothassunk különböző méretű, struktúrájú gráfokon, amennyiben a gráf megfelel a probléma kényszereinek.

2025/2026. 2. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1 Bevezető

A stabil párosítás (1), mint problémacsalád sok helyzetben fordulhat elő. Olyan problémák, mint a diákok egyetemre felvétele, orvosi rezidensek elhelyezése, beteg-donor párok meghatározása mind modellezhető stabil párosítási problémaként, ahol a jelentkezők és a jelentkezést váró felek gráfok csúcsaiként, egy konkrét jelentkezés pedig a csúcsok közötti élekként jelennek meg. A jelentkezők nem csak a jelentkezéseiket adják le, hanem ezeket a jelentkezéseket rangsorba is helyezik. Ez a rangsor egy jelentkező preferencialistája. Az előbb meghatározott problémák stabilitására vonatkozó feltételek hasonlóak, vagyis a párosításban nem szerepelhetnek blokkoló élek. A blokkoló élek a preferenciák és kényszerek alapján határozhatóak meg. A problémák közötti különbségek a gráfokra szabott kényszerek és a kialakítható párokban rejlenek. Vannak kényszerek, amelyek mellett ismert olyan polinomiális idejű algoritmus, ami képes megadni a stabil párosítást, ha létezik, és megadni azt is, ha nem létezik stabil párosítás, viszont olyan kényszerek is léteznek, amelyekre a probléma NP-teljes (2). A projekt célja tehát az, hogy létrejöjjön egy olyan eszköz, aminek segítségével tetszőleges kényszerek mellett hasznos információt lehessen biztosítani egy párosítási probléma megoldásáról. A megoldásnak eszköze egy GNN (Graph Neural Network), pontosabban egy GAT (Graph Attention Network) (3), aminek segítségével két módszer is elképzelhető (4) (5). Az egyik az, hogy magát a megoldást reprezentáló gráfot legeneráljuk, a másik pedig az, hogy a gráf éleire valószínűségeket határozunk meg attól függően, hogy a blokkoló élek osztályába tartozik, vagy sem. A második módszer egyszerűbbnek és megközelíthetőbbnek bizonyul, így erre esett a választásom. A modell feladata tehát gráfok éleinek osztályozása. Ehhez szükség lesz egy olyan problémára, amire ismert polinomiális algoritmus, annak érdekében, hogy a tanító adatainkat felcímkézhessük, és végezhesünk rajta felügyelt tanulást. A választott probléma a Stabil házasság. Erre a problémára létezik polinomiális algoritmus, aminek a neve Gale-Shapley algoritmus (1). A modell tehát Stabil házasság problémán keresztül fogja megtanulni a párosítási problémákban található blokkoló élek fogalmát.

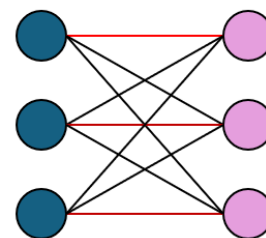
1.2 Elméleti összefoglaló

Az elméleti összefoglalóban mindenképpen szót kell ejtenem a párosításokról, azok stabilitásáról, a Stabil házasságról és a felhasznált mesterséges intelligencia modell működéséről és tanításáról.

Stabil párosítások és Stabil házasság

Vegyük a Stabil házasság problémát (1). Ennél a problémánál férfiak és nők akarnak házasodni egymással. A férfiak és nők száma megegyezik, és férfi csak nővel és nő csak férfivel házasodhat. Minden férfi és minden nő sorrendbe helyezi az ellenkező nemet, majd ezen preferenciák alapján kell megtalálni azt a párosítást, ahol senki sem elégedetlen. Akkor elégedetlen valaki, ha ő és egy másik ellenkező nemű csúcs is jobban preferálná egymást a jelenlegi partnerük helyett.

Formális definíció: Legyen $G = (V, E)$ páros gráf. V a párosítás szereplői, E a gráf színcsoportjai közötti lehetséges párok. Minden $v \in V$ csúcsához adott az ő preferenciáját leíró $\leq_v$ lineáris rendezés. V számára az e él jobb az f élnél, ha $f <_v e$. A G gráf egy M párosítása mellett $uv \notin M$ él akkor blokkoló él, ha



1. ábra. Egy Stabil házassági probléma gráfja.

u és v is jobban preferálja az uv élt a jelenlegi M -beli élek helyett. Ha egy M párosításban nincs blokkoló él, akkor stabil.

A stabilitás és blokkoló él fogalma általános. Ha elvetjük a páros gráf kényszert akkor megkapjuk a Stabil szobatárs problémát, és további kényszerek elhagyásával vagy hozzáadásával új problémákat kapunk. Attól függetlenül, hogy milyen kényszereink vannak, a stabilitás és blokkoló él fogalma alkalmazható ezekre a párosítási problémákra. (2)

Gráf Neurális Hálók és Graph Attention Network

A GNN-ek eltérnek a klasszikus neurális hálóktól (6) mind az általuk használt adatok struktúrájában, mind az őket felépítő rétegek működésének szempontjából. Egy klasszikus neurális háló bemeneti rétegből, rejtett rétegekből és egy kimeneti rétegből áll. Ezeket a rétegeket neuronok építik fel, amik a számítások elvégzéséért felelnek. Egy neurális háló életének két fázisa van: a tanulási fázis és az előhívási fázis. A tanulási fázis alatt tanító adatok segítségével a háló oly módon módosítja neuronjait, hogy azok később az előhívási fázisnál képesek legyenek az adatokban rejlő információt felismerni és használni.

Ezeket az előhívott felismeréseket különböző feladatokra használhatjuk. Ilyen feladatok például az osztályozás, ahol a bemeneti adatok alapján előre meghatározott osztályokba helyezzük az adatainkat, vagy akár használhatjuk a hálót regresszióra, ahol nem osztályokat, hanem konkrét számértékeket prédikálunk a bemeneti adatainkhoz. A projekt esetében osztályozás a használatos, így ezt az esetet ragadom meg és mutatom be.

Az osztályozásból két féle létezik: nem felügyelt és felügyelt osztályozás. Nem felügyelt osztályozásnál a modell saját maga alakítja ki az osztályokat az adatok jellemzői alapján. A projekt esetében előre tudjuk, hogy két osztályunk lesz, a blokkoló és nem blokkoló élek, így felügyelt osztályozást végzünk. Ebben az esetben előállítunk osztályozni kívánt példányokat, jellemzőket adunk meg hozzájuk és végül megadjuk, hogy melyik osztályba tartoznak.

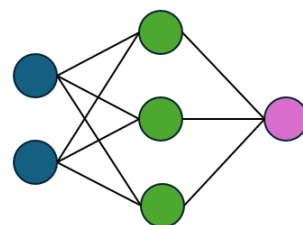
Az így megalkotott adatokat egymás után betápláljuk a hálónak, majd a végén megvizsgáljuk, hogy jól döntött-e, vagy sem. A döntések helyessége alapján létrehozhatunk egy függvényt, ami azt indikálja, mekkora hibákat ejtünk a kiértékelések során. Ezt a függvényt veszteség függvénynek hívják. Ezen függvénnyel képesek vagyunk úgy állítani a neuronok paramétereit, hogy az később kevesebbet hibázzon. A paraméterek állításának módját a veszteség függvény segítségével tudjuk meghatározni.

Célunk az, hogy a veszteség függvényt minimalizáljuk. A veszteség függvény globális minimumában követjük el a legkevesebb hibát, így ennek a megtalálása a célunk. Erre a feladatra több megközelítés is létezik, viszont a projekt keretében az ereszkedő gradiens algoritmust használom. Az ereszkedő gradiens a veszteség függvény elsőrendű deriváltját használja arra, hogy minden iterációban a derivált mértékével és megfelelő irányával elmozdítsa a modell paramétereit a veszteség minimuma felé. Az algoritmusnak így két lépése lesz: a hiba kiszámítása és a paraméterek frissítése. Matematikai képlettel az algoritmust így definiáljuk:

$$\theta_j = \theta_j - \alpha \frac{\partial C(\theta)}{\partial \theta_j}$$

A képletben θ_j a modell egy paraméterét jelöli, α az általunk megadott tanulási rátát, $C(\theta)$ az általunk definiált veszteség függvényt a modell jelenlegi paramétereinek mellett.

A Gráf Neurális Hálók az klasszikus neurális hálókkal szemben nem egyszerű szám inputokat használnak feladataikhoz, hanem gráfokat. Ezeket a gráfokat mátrixos reprezentációkba alakítjuk, majd így adjuk meg a hálónak. A gráfok természetéből adódóan nem csak a csúcsok jellemzői, hanem azok topológiája is információt hordoz. Egy klasszikus



2. ábra. Egy neurális háló felépítése

neurális háló nem lenne képes feldolgozni ezt az információt, így ennek megfelelően a rejtett rétegeket meg kell változtatni.

A projekt konkrét modellje a GAT (3). Egy GAT-ban a bemeneti rétegben megadjuk a gráf csúcsait, azoknak jellemzőit, a gráf éleit és azok jellemzőit. A bemeneti réteg után kerül a gráf egy GAT rétegbe, ahol három fő lépést végzünk el. A jellemzőket transzformáljuk egy tanulható súlymátrixszal, a jellemzők alapján a csúcsok fontosságát társítanak minden szomszédjukhoz, majd az utolsó lépésben a fontosságok alapján újra számítjuk minden csúcs reprezentációját a szomszédok információja alapján. Az eddig tárgyalt különbségektől eltekintve a tanítási és előhívási fázis megegyezik egy klasszikus neurális hálóéval.

A modell teljesítményének vizsgálata

Egy betanított modell teljesítményét a feladathoz megfelelően kell vizsgálnunk. A projekt esetében osztályozást végzünk két osztállyal, így első opciónak az élekre prédikált osztályok pontossága jön szóba. Ez a megközelítés viszont nem feltétlen tartalmaz elég információt arról, hogy a modellem a stabil párosítások megalkotásánál milyen jól teljesít. A feladathoz illően definiáltam egy egyszerű mérőszámot, ami különböző gráfok kiértékelése után párosításokat alkot mohó módon. A modell által legmegfelelőbbnek gondolt éleket egymás után válogatjuk be addig, amíg nem érünk el egy teljes párosításhoz. A teljes párosítás azt jelenti, hogy nincs olyan csúcs, akinek le lenne párja. Miután megalkottunk egy gráfra egy ilyen párosítást megvizsgáljuk a stabilitását. Ha a párosítás stabil, akkor jól párosítottuk a gráfot, ha pedig van benne blokkoló él, akkor rosszul párosítottuk. Ezek alapján egy stabilitási pontosságot számolhatunk a jól párosított gráfok és az összes gráf arányában.

Adatelemzési módszerek a tanítás javításához

A modelleknél tárgyalt tanítási módszerek sokszor nem bizonyulnak elégnek a modell végső teljesítményét vizsgálva. Segítségül hívhatunk különböző módszereket a pontosság javításának érdekében. Ilyen módszerek például a bemeneti jellemzők normalizálása, a gráfok struktúrájának manipulálása, a tanítás közbeni validáció alapján korai leállítás, a veszteség függvény újra definiálása, a tanító adatok osztályainak súlyozása és a végső párosítás megalkotásának finomítása. A modell egy határon belül szabadon paraméterezhető, bővíthető és strukturálható.

Probléma megoldásának menete

A szükséges információk átadása után összerakhatjuk a megoldás menetét. Első lépésünk a tanító adatok gyűjtése lesz. Megfelelő publikus adathalmaz hiányában sajátot kell alkotnom. Véletlenszerű gráfokat generálok a Stabil házasság probléma kényszerei mellett, majd statisztikai jellemzőket társítok a csúcsokhoz és élekhez a véletlenszerű preferenciák alapján. Ilyen statisztikai jellemzők például egy csúcs átlagos rangja. Ha elkészült egy gráf, akkor még osztályoznom kell az éleit blokkoló vagy nem blokkoló léte alapján. Ehhez a Gale-Shapley algoritmust (1) hívom segítségül. Megalkotom vele a stabil párosítást a gráfra, majd hozzárendelem a gráf éleihez az osztályát az algoritmus outputja alapján. Ha megvannak az adataim akkor a modell tanítása következik. A modellet tanítási fázisba helyezem, és addig tanítom amíg érdemleges tanítás történik a veszteség függvény alapján. Ha a veszteség függvény két tanítási lépés között nem változik egy ϵ értékkel legalább, akkor leállítom a tanítást. A tanítást követően kiértékelem a modell pontosságát.

1.3 A munka állapota, készültségi foka a félév elején

A munkában nem volt kiindulási pont, a projekt a tárgy keretében született meg.

2. Az elvégzett munka és eredmények ismertetése

2.1 Az adathalmaz megalkotása

A mélytanulási feladatok elengedhetetlen része a tanításra használt adathalmaz. Az elméleti összefoglalóban is már említett Stabil házasság problémának megfelelő kényszerekkel kell létrehozni a gráfokat. Az adatok létrehozásának első lépése a gráfok csúcsainak és éleinek megalkotása. A gráf adatstruktúrájáért NetworkX Python könyvtár felel. Ebben a könyvtárban felvehetünk gráfoknak csúcsokat, éleket és egyéb jellemzőket, mint például a csúcsok színosztályát. Páros gráfokat alkotok, minden csúcsból vezet él a másik csoport minden csúcsába. Az alapvető struktúra megalkotása után minden csúcshoz preferenciákat kell készítenem. Ezek a preferenciák fogják meghatározni a stabil párosítást, a csúcsok jellemzőit és az élek jellemzőit is. A jellemzők minden csúcshoz a következők: a másik csoportból hányszor szerepel elsőként a preferencialistákban, mi az átlag rangja, hányszor szerepel utolsóként, mi a mediánja a rangjainak. Az élek jellemzőit úgy számítom, hogy a két végpontjának rangjait egymásról összeadom.

A jellemzők elkészítése után egy külön függvény felel azért, hogy a jellemzők formátumát a modell bemenetéhez igazítsa. Az igazítás után már készen állok arra, hogy a modell dolgozhasson az adaton.

A preferenciák véletlenszerű generálása nem vezet a legjobb eredményre. Abban az esetben amikor véletlenszerűek a preferenciák, nagyon hasonló jellemzőket fognak felvenni az egyes csúcsok. Ilyenkor erős verseny alakul ki, vagyis csak nagyon minimális az eltérés egy blokkoló és nem blokkoló él között. Annak érdekében, hogy a modell tanulás alatt képes legyen felismerni az élek típusait, különböző előre megadott preferencia mintákat kell alkalmaznom. Egy hatásos minta az, hogy explicit megadott párok is létezzenek a gráfban. Ezt a mintát könnyű mintának hívom. Ebből a modell képes rátanulni arra, hogy a közeli preferenciákkal és jellemzőkkel rendelkező csúcsok egymással lesznek párosítva. A másik nagyon hasznos minta az, amikor pont az ellenkezőjét adjuk meg a könnyű mintának. Ekkor jól elkülöníthető, de élesen versenyző szituációk fognak kialakulni. Ennek a mintának a nehéz minta nevet adtam.

2.2 A modell megalkotása

Az adathalmaz rendelkezésre állása után megalkothatom a modellt. A modell első iterációja egy GAT modell volt, viszont tapasztalatok alapján a GATv2 (Graph Attention Network version 2) (7) jobban teljesít az adathalmazon. A GATv2 lényegében nem tér el a sima GAT modelltől. A fő különbség az üzenetátadási rejtett rétegen belüli fegyelmi fejeknél figyelhető meg. A klasszikus GAT statikus fegyelmi fejeket használ, míg a v2 dinamikus fejeket használ. Ezek a dinamikus fejek kifejezőbbek, vagyis bonyolultabb mintákat képesek felismerni és megtanulni. Az én esetemben ez előnyös, hiszen a párok megalkotása nagy mértékben függ attól, hogy a csúcsok mennyire figyelnek egymásra. A modell ezeket a GATv2Conv (Graph Attention Network version 2 Convolution) rétegeket használja a rejtett rétegében, ebből konkrétan kettőt. A felépített modell így néz ki:

$$\text{GATv2Conv} \rightarrow \text{Skip} \rightarrow \text{GATv2Conv} \rightarrow \text{Skip} \rightarrow \text{MLP (Multi-Layer Perceptron)}$$

Tehát a modell újra számítja a csúcsok reprezentációit, majd hozzáadja az eredeti reprezentációhoz (Skip). Ezt a folyamatot megismétli, majd egy végső MLP rétegen megy keresztül.

Az MLP réteget meg lehet feleltetni egy klasszikus neurális hálózatnak, ami arra használatos, hogy a reprezentációkból végső valószínűségeket számítson. Ebben az MLP-ben

egyszerű lineáris rétegeket, Dropout rétegeket és hozzájuk Tanh aktivációs függvényeket rendeltem.

A modell hiperparamétereit kézi finomítás után állítottam be. Minden GATv2Conv nyolc fegyelmi fejjel rendelkezik, és a rétegek Dropout paramétere 0,1. A Dropout paraméter azért felel, hogy a modell paramétereit egyenletesebben változzanak tanítás közben. Dropout nélkül hajlamos egyes paramétereket kihagyni a tanításból.

Végül a tanítási folyamat működtetéséhez az ADAGRAD (Adaptive Gradient) optimalizálót választottam. Ismét csak tapasztalat alapján ehhez az optimalizáléhoz 0,015 értékkel választottam meg a tanulási rátát.

A modell rétegeit a PyTorch Python könyvtárral alkottam meg. A PyTorch lehetőséget ad arra, hogy processzor helyett videókártyát használjak, így Anaconda segítségével létrehoztam egy környezetet, amivel képes vagyok videókártyán futtatni a modellt.

2.3 A tanítási folyamat

A modell és adathalmaz elkészülése után elkezdhetem a tanítást. A tanító adathalmaz 400 gráfból áll, ennek a fele könnyű a másik fele pedig nehéz minta. A gráfok színcsoportjai három csúcsból állnak, így minden gráfban hat csúcs és kilenc él található. A validációs adathalmaz 200 gráfból áll pontosan ugyan azzal a felépítéssel, mint a tanító halmaz.

A tanítás első lépése, hogy tanító adaton előre jelez a modell. Az előrejelzést használva kiszámolom a hibát. A hibát visszaterjesztem a modell paramétereire, ezzel javítva azokat. A javítás után a validációs halmazon teljesítményt számolok. Ha a teljesítmény növekedés elhanyagolható, akkor leállítjuk a tanítási folyamatot, abban az esetben pedig, ha a küszöb felett javult, akkor tovább tanulunk. A tanítás befejezése után elmentem a legjobb modellt, hogy azt később szabadon használhassam tesztelésre.

A veszteség számítása nagyon fontos szerepet kap a tanítás alatt. Kétszintű osztályozási feladatokra gyakran bináris keresztentropiát használnak, ami azt mutatja meg, mekkora eséllyel tévesztünk osztályt. Legyen p_{uv} a modell által prédikált osztály valószínűség egy élre. A p_{uv} érték az élekre azt fogja megmutatni, hogy a modell szerint mekkora valószínűséggel fog szerepelni az adott él a stabil párosításban. A bináris keresztentropiát a prédikált y címke és ennek a prédikációnak a p_{uv} valószínűségéből állítjuk elő.

A projekt természetéből adódóan viszont szükségem van még olyan tagokra is, amik a stabilitást és a párosítás teljességét büntetik. A pontos modell egy élt kell rendeljen minden csúcshoz, és lehetőleg olyan él legyen, ami nem blokkoló. Azt, hogy mindig csak egy él legyen kiválasztva, egyszerűen abból számoljuk és úgy büntetjük, hogy egy csúcs éleinek p_{uv} valószínűségeit összeadjuk majd ennek az összegnek az egy feletti részét vesszük az ebből származó veszteségnek. A blokkoló él választásának büntetését pedig az alapján számoljuk, hogy a nem kiválasztott, tehát nullához közeli p_{uv} valószínűségű élek közül van-e olyan, amit az él végei preferálnának a jelenlegi helyett. A jobban preferált élek megtalálását egy olyan $[u \text{ preferálja } v]$ függvény állítja elő, ami a blokkoló él definíciója alapján van implementálva. A döntés szigorúságának határát egy τ taggal állíthatom. Minden ilyen megtalált él valószínűségének komplementerét hozzáadjuk a veszteséghez.

Ennek a két tagnak a súlyát egy λ_1 és egy λ_2 értékkel állíthatom annak érdekében, hogy tanulás közben a megfelelő mennyiséggel szóljanak bele a veszteségbe. Ez azért szükséges, mivel a két új tag magában nem képes teljes egészében jellemezni a problémát, a veszteség fő motorja a bináris keresztentropia kell maradjon. A λ_1 és λ_2 súlyok megfelelő paraméterkereső keretrendszer hiányában 0,1 és 0,2 körüli értékeknél optimálisak manuális tesztek alapján.

A veszteséget így három tagból állítom elő (8):

$$\text{veszteség} = \text{bináris keresztentropia} + \lambda_1 * \text{párosítási korlát} + \lambda_2 * \text{stabilitási veszteség}$$

$$\text{bináris keresztentropia} = -(y * \log(p_{uv}) + (1 - y) * \log(1 - p_{uv}))$$

$$\text{párosítási korlát} = \sum_{u \in U} \max(0, \sum_v p_{uv} - 1)^2 + \sum_{v \in V} \max(0, \sum_u p_{uv} - 1)^2$$

$$\text{stabilitási veszteség} = \sum_{(u,v) \notin M} \mathbf{1}[u \text{ preferálja } v] \cdot \mathbf{1}[v \text{ preferálja } u] \cdot (1 - p_{uv})$$

Az újra definiált veszteség hatásfoka az 1. táblázatban látható. Kisebb gráfokra nem túl jelentős a javulás, de nagyobb esetekre a plusz stabilizáció egész sokat segít.

	Bináris keresztentropia	Három tagú veszteség
n = 6	71%	74%
n = 8	68%	69%
n = 10	25%	30%

1. táblázat. Modell pontossága különböző gráfméretekre bináris keresztentropia és összetett veszteség esetén

2.4 A teljesítmény javítása

Az 1. táblázatban elért eredményekhez szükség volt még pár technikára. Ilyen technika például a normalizáció. Bemeneti adataimban előfordul, hogy kiugró értékek szerepelnek. Annak érdekében, hogy ezek a kiugró értékek ne befolyásolják negatívan a tanítást, normalizálom őket a többi adathoz képest. Ebben a projektben a z-score normalizációt választottam, ahol az adatokat az adathalmazból számított átlaggal és szórással nulla körüli átlagúvá és 1 körüli szórásuvá teszem. Ez segít stabilizálni a tanulási folyamatot, és jelentősen javítja a pontosságot. Ha a normalizációt eltávolítom a tanítási és előhívási fázisból a modell közel 40%-os pontosságvesztést mutat.

A fejlesztés alatt kipróbáltam különböző technikákat a normalizáció mellett, viszont ezek nem vezettek eredményre.

Végül az állítható paraméterek finomhangolását végeztem el. Az implementáción nem csak a modell paramétereit tudom állítani, hanem még a veszteség paramétereit, a gráfok preferencialistáit, a korai leállítási paramétereit és a gráfok méreteit is. Ezeket mind kézzel hangoltam be a teszteredmények alapján. Mivel nagyon sok és különböző természetű paraméter van nem találtam előre elkészített paraméterkereső keretrendszert, ami megfelelt volna igényeimnek.

2.5 Párosítások megalkotása és tesztelés

A végső betanított modell elkészült. Az utolsó dolog, amit még el kell végezni az a modell jószágának mérése.

Az elméleti összefoglalóban már említett módon a modell a párosítás megalkotásának csak az első lépése. Miután megadott egy előrejelzést a modell a gráfra ki kell választanom a legjobb éleket. Erre a feladatra egy egyszerű mohó kiválasztást implementáltam. A mohó algoritmus minden ciklusában kiválasztja a legnagyobb valószínűségű élt mindaddig, amíg nem keletkezik teljes párosítás. Egy másik lehetőség a kiválasztásra az lett volna, hogy minden kiválasztás után új gráfot alkotunk a megmaradt csúcsokból és preferenciáikból, viszont ez jelentős romláshoz vezetett a modell pontosságánál. Ha ezt a módot választottam volna, akkor minden kivételnél fontos információ vesz el a párosításról.

Ezzel a kiválasztási algoritmussal már képes vagyok párosításokat alkotni, így lemérhetem a modellt. Mielőtt a modell eredményeit értelmezném érdemes felállítani egy alap szintet a pontosságra nézve. Bináris osztályozási feladatoknál a véletlen osztályozót szokás alapnak tekinteni, ahol az algoritmus $\frac{1}{2}$ valószínűséggel osztja be egy osztályba az alanyt. A véletlen osztályozó teljes párosításokat választana ki véletlenszerűen. Ekkor a valószínűsége annak, hogy eltaláljuk a stabil párosítást, egy olyan gráfra, amiben n csúcs van, színosztályonként $\frac{n}{2}$ csúcs, legalább $\frac{1}{\left(\frac{n}{2}\right)!}$. Fontos megemlíteni, hogy egy Stabil házassági problémában mindig létezik

legalább egy stabil párosítás, viszont létezhet több is. Annak okán, hogy több stabil párosítás is létezhet, csak alsó és felső határt tudunk megadni a véletlen osztályozó pontosságára. Ha a preferencia lista megenged több stabil párosítást, akkor ezeknek a számossága lenne a felső korlát és a pontosság az előbb említett képlettel lenne számolva. Mivel nem ismert pontos felső korlát (9), ezért csak az alsó korláttal tudunk számolni megbízhatóan.

A továbbiakban feltételezem tehát, hogy a tesztleinkben csak egy stabil párosítás létezik. Ekkor a véletlen osztályozónk a képlet alapján 16,67%-os pontosságot képes produkálni $n = 6$ csúcsnál.

A modellt 1000 darab $n = 6$ csúcsú gráfra tesztelem. A tesztesetek fele könnyű minta a másik fele pedig nehéz minta. Ezek a gráfok a rájuk vonatkozó kényszerek és saját magam által felállított szabályoktól eltekintve véletlenszerű preferenciákkal rendelkeznek. A teszt többszöri lefutása után és az eredményeket átlagolva a modell pontossága 72% körül mozog. Ha elhagyjuk a mintáinkat és a preferenciák teljesen véletlenszerűen vannak megadva, akkor a modell pontossága 35% körül mozog.

Ezeket a pontosságokat úgy állítottam elő, hogy az 1000 darab gráfból hány gráfra sikerült megadni a stabil párosítást, majd ezt a számot elosztottam az összes gráf számával.

Az eredményeket fontos értelmezni. A legnehezebb esetben is sikerül jobb pontosságot elérni, mint a véletlen osztályozó pontossága, viszont nagy különbség van a véletlenszerű tesztek és a minták tesztjei között. Ha megnézzük a modell által generált valószínűségeket azokra az esetekre, amikor nem sikerült megalkotni a stabil párosítást észrevehetjük, hogy ezeknél az eseteknél a preferencia listák miatt éles verseny alakul ki. Nem találhatóak egyszerűen megadható párok és a csúcsok jellemzői között vagy minimális az eltérés, vagy nincs is eltérés. Mivel a modell az üzenetátadási rétegnél a jellemzők alapján alakít párokat az eltérések alapján ezért itt megbukik a modell. A jellemzők egyszerű statisztikai mérőszámok, így, ha a csúcsok preferencialistái megegyeznek akkor az őket jellemző statisztikai mérőszámok is megfognak egyezni.

Az eredmények tehát azt mutatják, hogy olyan esetekben, ahol jól elkülöníthető párok léteznek képes felismerni a modell a blokkoló éleket.

Mivel a stabil párosítás a kényszerektől függetlenül létezik, ezért semmi sem tart vissza attól, hogy más kényszerek mellett ki lehessen próbálni a modellt. A Stabil szobatárs problémát megkaphatjuk úgy, hogy a Stabil házasság problémából elvesszük a páros gráf kényszerét. Ebben az esetben tehát nincsenek színosztályok, bármelyik csúcs alkothat párt bármelyik csúccsal. Az adatok generálását elvégeztem és kipróbáltam a modellt rajtuk.

A modell teljesen megbukik ebben az esetben. A prédikált értékeket szemügyre véve megállapítható, hogy minden esetben egy valószínűségre fog konvergálni az összes előrejelzés. A probléma abból ered, hogy a modell a tanulás közben megtanulja, hogy minden szomszédja egy másik csoportban lévő elem. A Stabil szobatárs esetében minden csúcs minden szomszédját egy csoportnak tekinti, aminek a végkifejlete az lesz, hogy mindenki mindenkit ugyanúgy jellemez, ezért konvergál egy értékre az eredmény.

Végezetül kipróbáltam LLM modelleket is a Stabil házasság problémán. Első modell a ChatGPT 5.4 lett. Ugyan azokat a tesztek adtam meg, mint a modellemnek. A pontossága saját ellenőrzés után 100%-os lett. Következő modell egy sokkal kisebb, lokálisan futtatható Mistral 7B lett. A modell teljesítménye közel 0%-os lett, 1000 tesztadaton nagyjából csak egy vagy kettő párosítást adott meg jól.

2.6 Összefoglalás

A projekt keretében Graph Attention Network modellt tanítottam Stabil házassági problémában jelen lévő blokkoló élek felismerésére. A modell által biztosított valószínűségek segítségével stabil párosításokat alkottam gráfokra.

A modell tanításához létrehoztam egy keretrendszert, ami a probléma kényszereinek eleget tevő gráfokat hoz létre.

Az adatok feldolgozásához és tanításához létrehoztam egy keretrendszert, ami GAT modellel és különböző adatelemzési módszerekkel képes a problémában rejlő matematikai struktúrát megtanulni és felismerni.

A Stabil házasság probléma egyszerűbb esetein 70% körüli pontosságot mutat, nehezebb eseteken viszont bizonytalan a modell a döntéseiben és csak 35%-os pontosságot képes elérni. Más hasonló problémákon a modell nem képes használható prédikációkat hozni.

A stabil párosítási problémacsalád több hasonló problémát foglal magába és valós helyzetekre kínál megoldást. Ezek a problémák egymástól csak a rájuk szabott kényszerekben különböznek, így a cél az volt, hogy egy olyan modellt alkossak, ami a kényszerektől függetlenül fel képes ismerni a blokkoló éleket. Egy ilyen modell megalkotása több olyan problémára is kínálna megoldást amire csak exponenciális algoritmus ismert. Erre a feladatra nem bizonyult jó megoldásnak a Stabil házasságra tanított modell.

A felvetett általános stabil párosítás megadó modell létrehozását nem zárják ki ezek az eredmények. További fejlesztések lehetnek a tanító adatok gazdagítása vagy lecserélése, különböző modellek létrehozása és kombinálása vagy az adatok jellemzőinek újra definiálása, vagy akár egy teljesen más megközelítés alkalmazása a tanulási fázis alatt.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

A tanulmányozott irodalom jegyzéke:

1. *College Admissions and the Stability of Marriage*. **Shapley, David Gale és Lloyd S.** 1, 1962., The American Mathematical Monthly, 69. kötet, old.: 9–15.
2. **Péter, Bíró.** The stable matching problem and its generalizations: an algorithmic and game theoretical approach. *BME Doktori Iskola vagy BME doktori repozitórium*. [Online] 2007. [Hivatkozva: 2026. április 2.] doktori.math.bme.hu/biro_tezisfuzet.pdf.
3. *Graph Attention Networks*. **Veličković, Petar, és mtsai.** International Conference on Learning Representations (ICLR) : ismeretlen szerző, 2018.
4. *Unsupervised learning with GNNs for QUBO-based combinatorial optimization*. **Krylova, Olga és Phillipson, Frank.** 2025., EURO Journal on Computational Optimization, 13. kötet, old.: 100116.
5. *Combinatorial Optimization and Reasoning with Graph Neural Networks*. **Cappart, Quentin, és mtsai.** 130, 2023., Journal of Machine Learning Research, 24. kötet, old.: 1-61.
6. **Goodfellow, Ian, Bengio, Yoshua és Courville, Aaron.** *Deep Learning*. Cambridge, MA : MIT Press, 2016.
7. *How Attentive are Graph Attention Networks?* **Brody, Shaked, Alon, Uri és Yahav, Eran.** International Conference on Learning Representations (ICLR) : ismeretlen szerző, 2022.
8. *Contrastive Learning for Supervised Graph Matching*. **Ratnayaka, Gathika, Wang, Qing és Li, Yang.** 2023., Proceedings of the 39th Conference on Uncertainty in Artificial Intelligence (UAI 2023), 216. kötet, old.: 1718–1729.
9. **Palmer, Cory és Pálvölgyi, Dömötör.** At most 3.55^n stable matchings. *arXiv*. [Online] 2021. arxiv.org/abs/2011.00915.

Csatlakozó egyéb elkészült dokumentációk / fájlok / stb. jegyzéke:

A projekt forráskódja elérhető a következő hivatkozáson: <https://github.com/csuri66/Maths-AI>